1. Control Structures in C

By default the statements of a C program are executed one after another, from top to bottom. A control structure changes that order. It lets the program choose between different paths, or repeat a group of statements.

Type	What it does	Statements
Sequence	Statements execute one after another in the written order	ordinary statements
Selection (decision)	Chooses one path out of two or more, based on a condition	if, if-else, else-if ladder, switch
Iteration (looping)	Repeats a group of statements while a condition holds	while, do-while, for
Jump	Transfers control to another place in the program	break, continue, goto, return

2. Decision Making Statements

A decision making statement tests a condition and decides which statements to execute. In C, a condition produces 1 when it is true and 0 when it is false. Any non-zero value is treated as true.

2.1 The if statement

The if statement executes a block only when the condition is true. If the condition is false, the block is skipped and the program continues with the next statement.

```
if (condition)
{
    statement block;
}
```

Figure 1: Flowchart of the if statement

- The braces can be left out when the block has only one statement, but using them is safer.
- A semicolon must not be placed after if (condition), because that makes the body empty.

Example

```
#include <stdio.h>

int main()
{
    int marks;

    printf("Enter marks : ");
    scanf("%d", &marks);

    if (marks >= 33)
        printf("Result : Pass\n");

    printf("Program over\n");
    return 0;
}
```

Output

```
Enter marks : 55
Result : Pass
Program over
```

2.2 The if-else statement

The if-else statement chooses between two blocks. If the condition is true the first block runs, otherwise the second block runs. Exactly one of the two always runs.

```
if (condition)
{
    statement block 1;
}
else
{
    statement block 2;
}
```


Figure 2: Flowchart of the if-else statement

Algorithm — check whether a number is even or odd

Step 1: Start
Step 2: Read the number num
Step 3: If num % 2 equals 0, then print "even" and go to Step 5
Step 4: Otherwise print "odd"
Step 5: Stop

Example

```
#include <stdio.h>

int main()
{
    int num;

    printf("Enter a number : ");
    scanf("%d", &num);
```

```
    if (num % 2 == 0)
        printf("%d is even\n", num);
    else
        printf("%d is odd\n", num);

    return 0;
}
```

Output

```
Enter a number : 17
17 is odd
```

2.3 Nested if

An if or if-else statement written inside another if or else block is called a nested if. It is used when a second condition has to be tested only after the first one has been decided.

```
if (condition 1)
{
    if (condition 2)
        statement 1;
    else
        statement 2;
}
else
{
    statement 3;
}
```

Algorithm — largest of three numbers

Step 1: Start
Step 2: Read three numbers a, b and c
Step 3: If a > b then go to Step 4, otherwise go to Step 5
Step 4: If a > c then print a as largest, otherwise print c as largest; go to Step 6
Step 5: If b > c then print b as largest, otherwise print c as largest
Step 6: Stop

Example

```
/* Largest of three numbers using nested if */
#include <stdio.h>

int main()
{
    int a, b, c;

    printf("Enter three numbers : ");
    scanf("%d %d %d", &a, &b, &c);

    if (a > b)
    {
        if (a > c)
            printf("Largest = %d\n", a);
        else
```

```
            printf("Largest = %d\n", c);
    }
    else
    {
        if (b > c)
            printf("Largest = %d\n", b);
        else
            printf("Largest = %d\n", c);
    }
    return 0;
}
```

Output

```
Enter three numbers : 25 40 15
Largest = 40
```

2.4 The else-if ladder

The else-if ladder tests several conditions one after another. The moment a condition is found true, its block runs and the rest of the ladder is skipped. If no condition is true, the final else block runs.

```
if (condition 1)
    statement 1;
else if (condition 2)
    statement 2;
else if (condition 3)
    statement 3;
else
    default statement;
```


Figure 3: Flowchart of the else-if ladder

Example

```
/* Grade of a student using else-if ladder */
#include <stdio.h>

int main()
{
    int marks;

    printf("Enter marks : ");
    scanf("%d", &marks);

    if (marks >= 75)
        printf("Grade : Distinction\n");
    else if (marks >= 60)
        printf("Grade : First Division\n");
    else if (marks >= 45)
        printf("Grade : Second Division\n");
    else if (marks >= 33)
        printf("Grade : Third Division\n");
    else
        printf("Grade : Fail\n");

    return 0;
}
```

Output

```
Enter marks : 68
Grade : First Division
```

The order of the conditions matters. If the ladder began with marks >= 33, every passing student would be given the third division, because that condition becomes true first.

3. The switch Statement

The switch statement selects one block out of many by comparing a single value against a list of constants. It is a cleaner alternative to a long else-if ladder when the choice depends on one variable.

```
switch (expression)
{
    case constant1 : statement block 1;
                     break;
    case constant2 : statement block 2;
                     break;
    ...
    default        : default block;
}
```

Figure 4: Flowchart of the switch statement

Rules of switch

- The expression must give an integer or a character value. float and double are not allowed.
- Every case label must be a constant, not a variable or an expression.
- No two case labels can have the same value.
- The case blocks may be written in any order.
- default is optional, and it runs when no case matches.
- break takes control out of the switch. Without it, execution falls through into the next case.

Example

```
/* Simple calculator using switch */
#include <stdio.h>

int main()
{
    int a, b;
    char op;

    printf("Enter two numbers  : ");
    scanf("%d %d", &a, &b);
    printf("Enter operator     : ");
    scanf(" %c", &op);
```

```
    switch (op)
    {
        case '+' : printf("Sum        = %d\n", a + b);
                   break;
        case '-' : printf("Difference = %d\n", a - b);
                   break;
        case '*' : printf("Product    = %d\n", a * b);
                   break;
        case '/' : printf("Quotient   = %d\n", a / b);
                   break;
        default  : printf("Invalid operator\n");
    }
    return 0;
}
```

Output

```
Enter two numbers  : 20 4
Enter operator     : *
Product    = 80
```

What happens when break is missing

If break is left out, control continues into the following cases even though their labels do not match. This is called fall through.

Example — switch without break

```
/* What happens when break is missing */
#include <stdio.h>

int main()
{
    int day = 2;

    switch (day)
    {
        case 1 : printf("Monday\n");
        case 2 : printf("Tuesday\n");
        case 3 : printf("Wednesday\n");
        case 4 : printf("Thursday\n");
        default: printf("Other day\n");
    }
    return 0;
}
```

Output

```
Tuesday
Wednesday
Thursday
Other day
```

The value 2 matched case 2, but because there is no break the program went on to print Wednesday, Thursday and the default block as well.

Difference between switch and else-if ladder

Point	switch	else-if ladder

Tested value	one integer or character expression	any condition, including ranges
Comparison	equality with constants only	any relational or logical expression
float values	not allowed	allowed
Speed	usually faster, a jump table can be used	conditions are tested one by one
break needed	yes, to stop fall through	no

4. Looping Statements

A loop repeats a group of statements while a condition remains true. Every loop needs three things: an initialization, a test condition, and an update that eventually makes the condition false. If the update is missing, the loop never ends and becomes an infinite loop.

Loop	Condition tested	Runs at least once	Best used when
while	before the body	no	the number of repetitions is not known in advance
do-while	after the body	yes	the body must run at least once, such as a menu
for	before the body	no	the number of repetitions is known in advance

4.1 The while loop

The while loop is an entry controlled loop. The condition is tested first, and the body runs only if it is true. If the condition is false at the very start, the body never runs.

```
initialization;
while (condition)
{
    body of the loop;
    update;
}
```

Figure 5: Flowchart of the while loop

Algorithm — sum of first n natural numbers

Step 1: Start
Step 2: Read n
Step 3: Set i = 1 and sum = 0
Step 4: Repeat Steps 5 and 6 while i is less than or equal to n
Step 5: Set sum = sum + i
Step 6: Set i = i + 1
Step 7: Print sum
Step 8: Stop

Example

```
/* Sum of first n natural numbers using while */
#include <stdio.h>

int main()
{
    int n, i = 1, sum = 0;

    printf("Enter n : ");
    scanf("%d", &n);

    while (i <= n)          /* test first, then execute */
    {
```

```
        sum = sum + i;
        i++;
    }

    printf("Sum of first %d numbers = %d\n", n, sum);
    return 0;
}
```

Output

```
Enter n : 10
Sum of first 10 numbers = 55
```

4.2 The do-while loop

The do-while loop is an exit controlled loop. The body runs first and the condition is tested afterwards, so the body always runs at least once. The closing while must end with a semicolon.

```
initialization;
do
{
    body of the loop;
    update;
} while (condition);
```

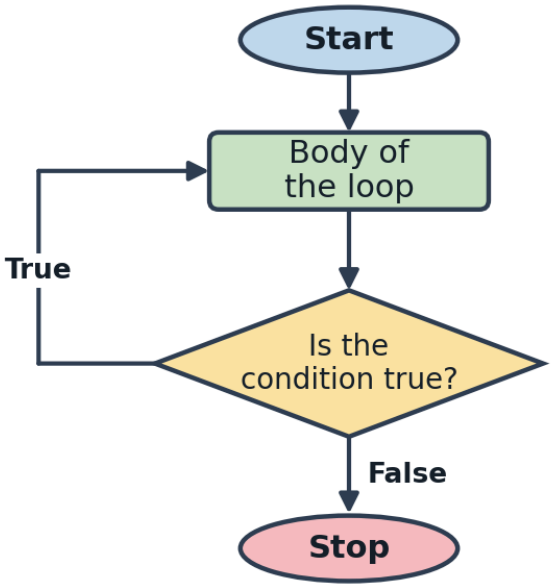


Figure 6: Flowchart of the do-while loop

Example

```
/* Reverse the digits of a number using do-while */
#include <stdio.h>
```

```
int main()
{
    int num, digit, rev = 0;

    printf("Enter a number : ");
    scanf("%d", &num);

    do                      /* body runs once before the test */
    {
        digit = num % 10;
        rev   = rev * 10 + digit;
        num   = num / 10;
    } while (num != 0);

    printf("Reversed number = %d\n", rev);
    return 0;
}
```

Output

```
Enter a number : 4056
Reversed number = 6504
```

The number is broken down one digit at a time: num % 10 takes the last digit, and num / 10 removes it.

while compared with do-while

Example — the same condition in both loops

```
/* while runs zero times, do-while runs at least once */
#include <stdio.h>

int main()
{
    int i = 10;

    printf("while loop  : ");
    while (i <= 5)
    {
        printf("%d ", i);
        i++;
    }
    printf("(nothing printed)\n");

    i = 10;
    printf("do-while    : ");
    do
    {
        printf("%d ", i);
        i++;
    } while (i <= 5);
    printf("\n");

    return 0;
}
```

Output

```
while loop  : (nothing printed)
do-while    : 10
```

The condition i <= 5 is false from the beginning. The while loop printed nothing, while the do-while loop still printed 10 once before testing.

4.3 The for loop

The for loop keeps the initialization, the condition and the update together in one line, which makes it convenient when the number of repetitions is known.

```
for (initialization; condition; update)
{
    body of the loop;
}
```

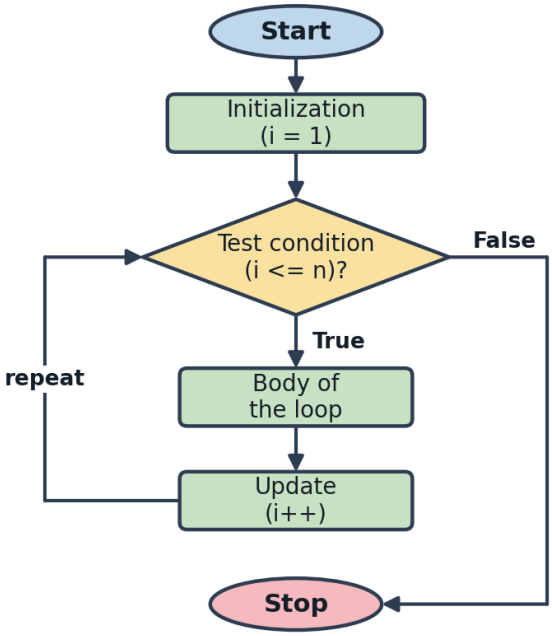


Figure 7: Flowchart of the for loop

- The initialization runs only once, at the start.
- The condition is tested before every repetition.
- The update runs after the body, at the end of every repetition.
- All three parts are optional. for (; ;) creates an infinite loop.
- More than one variable can be handled using the comma operator, as in for (i = 0, j = 10; i < j; i++, j--).

Example

```
/* Multiplication table using for */
#include <stdio.h>

int main()
{
```

```
    int n, i;

    printf("Enter a number : ");
    scanf("%d", &n);

    for (i = 1; i <= 10; i++)
        printf("%d x %2d = %3d\n", n, i, n * i);

    return 0;
}
```

Output

```
Enter a number : 7
7 x  1 =   7
7 x  2 =  14
7 x  3 =  21
7 x  4 =  28
7 x  5 =  35
7 x  6 =  42
7 x  7 =  49
7 x  8 =  56
7 x  9 =  63
7 x 10 =  70
```

4.4 Nested loops

A loop written inside another loop is called a nested loop. For every single repetition of the outer loop, the inner loop completes all of its repetitions.

Example — star pattern

```
/* Star pattern using nested for loops */
#include <stdio.h>

int main()
{
    int rows = 5, i, j;

    for (i = 1; i <= rows; i++)        /* outer loop : rows    */
    {
        for (j = 1; j <= i; j++)       /* inner loop : columns */
            printf("* ");
        printf("\n");
    }
    return 0;
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
```

The outer loop controls the rows and the inner loop controls how many stars appear in each row. Because the inner condition is j <= i, row number i prints i stars.

5. break, continue and goto

5.1 The break statement

break ends the loop immediately and control moves to the first statement after the loop. Inside a switch, break ends the switch. It cannot be used outside a loop or a switch.

```
break;
```

Example

```
/* break leaves the loop completely */
#include <stdio.h>

int main()
{
    int i;

    for (i = 1; i <= 10; i++)
    {
        if (i == 5)
            break;          /* loop ends here */
        printf("%d ", i);
    }
    printf("\nLoop ended at i = %d\n", i);
    return 0;
}
```

Output

```
1 2 3 4
Loop ended at i = 5
```

The numbers 1 to 4 were printed. When i became 5 the break ended the loop, so 5 itself was never printed.

5.2 The continue statement

continue skips the remaining statements of the current repetition and goes on to the next one. The loop itself keeps running. In a for loop the update part still takes place.

```
continue;
```

Figure 8: break leaves the loop, continue skips only the current round

Example

```
/* continue skips the rest of this round only */
#include <stdio.h>

int main()
{
    int i;

    for (i = 1; i <= 10; i++)
    {
        if (i % 2 == 0)
            continue;       /* skip the even numbers */
        printf("%d ", i);
    }
    printf("\nLoop completed\n");
    return 0;
}
```

Output

```
1 3 5 7 9
Loop completed
```

Every even value of i met the continue, so its printf was skipped, but the loop went on until i reached 10.

Difference between break and continue

Point	break	continue
Effect	ends the loop completely	skips the rest of the current repetition
Control goes to	the first statement after the loop	the next repetition of the same loop
Used in	loops and switch	loops only
Remaining repetitions	are not performed	are performed normally

5.3 The goto statement

goto transfers control directly to a labelled statement in the same function. A label is a name followed by a colon.

```
goto label;
...
label:
    statement;
```

Example

```
/* goto jumps to a labelled statement */
#include <stdio.h>

int main()
{
    int i = 1;

start:                      /* label */
    printf("%d ", i);
    i++;
    if (i <= 5)
        goto start;         /* jump back to the label */

    printf("\nDone\n");
    return 0;
}
```

Output

```
1 2 3 4 5
Done
```

- goto can jump forward or backward within the same function.
- It cannot jump into or out of another function.
- Heavy use of goto makes a program hard to read and to debug. Such code is often called spaghetti code.
- Almost every use of goto can be replaced by a loop, an if-else, or break and continue, which is the preferred style.

6. Functions in C

6.1 What is a function?

A function is a self-contained block of statements that is given a name and performs one particular task. Once it is written, the task can be carried out any number of times simply by writing the name of the function. This is called calling the function. Every C program already uses functions. printf() prints, scanf() reads, and main() is itself a function, the one where execution begins.

6.2 Why do we need functions?

The clearest way to see the need is to write the same program twice. Suppose the area of three circles has to be printed.

Without a function

```
/* The same calculation written again and again */
#include <stdio.h>

int main()
{
    float r1 = 2.0, r2 = 3.5, r3 = 5.0;
    float a1, a2, a3;

    a1 = 3.14159 * r1 * r1;          /* repeated */
    printf("Area 1 = %.2f\n", a1);

    a2 = 3.14159 * r2 * r2;          /* repeated */
    printf("Area 2 = %.2f\n", a2);

    a3 = 3.14159 * r3 * r3;          /* repeated */
    printf("Area 3 = %.2f\n", a3);

    return 0;
}
```

Output

```
Area 1 = 12.57
Area 2 = 38.48
Area 3 = 78.54
```

The same formula is typed three times. If the value of π has to be corrected, or the formula changed, it must be corrected in three places, and missing one of them creates a bug that is hard to find.

With a function

```
/* The same work, written once inside a function */
#include <stdio.h>

float area(float r);                 /* declaration */

int main()
{
    printf("Area 1 = %.2f\n", area(2.0));
```

```
    printf("Area 2 = %.2f\n", area(3.5));
    printf("Area 3 = %.2f\n", area(5.0));
    return 0;
}

float area(float r)                  /* written once, used three times */
{
    return 3.14159 * r * r;
}
```

Output

```
Area 1 = 12.57
Area 2 = 38.48
Area 3 = 78.54
```

The formula is now written once. The three calls use it three times, and the output is exactly the same. Changing the formula means changing one line.

Advantages of using functions

- Avoids repetition — code written once can be called any number of times.
- Divides the work — a large program becomes a set of small, manageable parts.
- Easier testing — each function can be checked and corrected on its own.
- Reusability — a function written for one program can be used in another.
- Readability — a call such as area(2.0) states what is happening, while the formula does not.
- Easier maintenance — a correction is made in one place instead of many.
- Less memory in the source — the statements are stored once, however many times they are used.

6.3 How a function call works

When a function is called, control leaves the calling function and moves to the called function. After the called function finishes, control returns to the statement just after the call, bringing the returned value with it.

Figure 9: Control moves to the function on a call and returns to the caller afterwards

6.4 Types of functions

C functions are of two kinds, depending on who wrote them.

Point	Library functions	User-defined functions
Written by	the designers of the C compiler	the programmer
Where they are	already present in header files	written in the program itself
Availability	ready to use after #include	must be written before being used
Can be changed	no	yes
Examples	printf(), scanf(), sqrt(), strlen()	area(), swap(), factorial()

Example — using library functions

```
/* Library functions come ready made */
#include <stdio.h>
#include <math.h>
#include <string.h>

int main()
{
    printf("sqrt(64)        = %.2f\n", sqrt(64));
```

```
    printf("pow(2, 5)       = %.2f\n", pow(2, 5));
    printf("strlen(\"BCA\")   = %zu\n", strlen("BCA"));
    return 0;
}
```

Output

```
sqrt(64)        = 8.00
pow(2, 5)       = 32.00
strlen("BCA")   = 3
```

sqrt() and pow() come from math.h and strlen() from string.h, so those headers must be included. On Linux, a program using math.h is compiled with the -lm option: gcc prog.c -o prog -lm

7. User-defined Functions

A user-defined function is one that the programmer writes. This section covers how such a function is declared, defined and called.

The three parts of a function

Part	Where it is written	Purpose
Function declaration (prototype)	before main(), or inside it	tells the compiler the name, the return type and the types of the arguments
Function call	inside main() or another function	transfers control to the function and passes the arguments
Function definition	usually after main()	contains the actual statements of the function

```
return_type function_name(parameter list);          /* declaration */

function_name(arguments);                           /* call        */

return_type function_name(parameter list)           /* definition  */
{
    body of the function;
    return value;
}
```

Categories of functions

Functions are grouped by whether they take arguments and whether they send a value back.

Category	Arguments	Return value
1	no	no
2	yes	no
3	yes	yes
4	no	yes

Category 1 — no arguments, no return value

```
/* Category 1 : no arguments, no return value */
#include <stdio.h>

void greet();               /* function declaration (prototype) */

int main()
{
    greet();                /* function call */
    printf("Back in main()\n");
    return 0;
}

void greet()                /* function definition */
{
    printf("Welcome to C programming\n");
}
```

Output

```
Welcome to C programming
Back in main()
```

Category 2 — arguments, no return value

```
/* Category 2 : arguments, no return value */
#include <stdio.h>

void showSum(int a, int b);

int main()
{
    showSum(15, 25);
    return 0;
}

void showSum(int a, int b)
{
    printf("Sum = %d\n", a + b);
}
```

Output

```
Sum = 40
```

Category 3 — arguments with a return value

```
/* Category 3 : arguments with a return value */
#include <stdio.h>

int square(int n);          /* returns an int, takes an int */

int main()
{
    int num = 6, result;

    result = square(num);
    printf("Square of %d = %d\n", num, result);
    printf("Square of 9 = %d\n", square(9));
    return 0;
```

```
}

int square(int n)
{
    return n * n;           /* value sent back to the caller */
}
```

Output

```
Square of 6 = 36
Square of 9 = 81
```

Category 4 — no arguments, with a return value

```
/* Category 4 : no arguments, but returns a value */
#include <stdio.h>

int getNumber();

int main()
{
    int n;

    n = getNumber();
    printf("You entered %d\n", n);
    return 0;
}

int getNumber()
{
    int x;
    printf("Enter a number : ");
    scanf("%d", &x);
    return x;
}
```

Output

```
Enter a number : 42
You entered 42
```

Points to remember

- A function can return only one value using return.
- void as the return type means the function returns nothing.
- The values sent in the call are called actual arguments; the ones received by the function are called formal parameters.
- The number, order and type of the arguments must match the declaration.
- Variables declared inside a function are local to it and are destroyed when the function ends.

8. Nesting of Functions

Nesting of functions means one function calling another function. C allows a function to call any other function, and that function may call a third one.
C does not allow a function to be defined inside another function. Only calling is nested, not the definitions. This is a point of difference from some other languages.

Example

```
/* Nesting of functions : one function calls another */
#include <stdio.h>

int square(int n);
int sumOfSquares(int a, int b);

int main()
{
    int result;

    result = sumOfSquares(3, 4);        /* main calls sumOfSquares  */
    printf("Sum of squares = %d\n", result);
    return 0;
}

int sumOfSquares(int a, int b)
{
    return square(a) + square(b);       /* which calls square twice */
}

int square(int n)
{
    return n * n;
}
```

Output

```
Sum of squares = 25
```

main() calls sumOfSquares(), which in turn calls square() twice. Control returns to the caller each time a called function finishes, and finally comes back to main().

9. Call by Value and Call by Reference

Arguments can be passed to a function in two ways. The difference lies in whether the function receives copies of the values, or the addresses of the original variables.

Figure 10: The function works on copies in call by value, and on the original variables in call by reference

8.1 Call by value

In call by value, the values of the actual arguments are copied into the formal parameters. The function works on the copies, so any change made inside the function does not affect the original variables. This is the default method in C.

Example

```
/* Call by value : the function works on a copy */
#include <stdio.h>

void swap(int x, int y);

int main()
{
    int a = 10, b = 20;

    printf("Before call : a = %d, b = %d\n", a, b);
    swap(a, b);
    printf("After  call : a = %d, b = %d\n", a, b);
    return 0;
}

void swap(int x, int y)
{
    int temp = x;
```

```
    x = y;
    y = temp;
    printf("Inside swap : x = %d, y = %d\n", x, y);
}
```

Output

```
Before call : a = 10, b = 20
Inside swap : x = 20, y = 10
After  call : a = 10, b = 20
```

x and y were exchanged inside the function, but a and b in main() stayed as they were, because the function only ever had copies.

8.2 Call by reference

In call by reference, the addresses of the actual arguments are passed. The formal parameters are pointer variables, and the function reaches the original variables through those addresses. Changes made inside the function are therefore permanent.

- The call sends addresses using the & operator, as in swap(&a, &b).
- The parameters are declared as pointers, as in void swap(int *x, int *y).
- Inside the function, *x means the value stored at that address.

Example

```
/* Call by reference : the function works on the originals */
#include <stdio.h>

void swap(int *x, int *y);      /* takes addresses */

int main()
{
    int a = 10, b = 20;

    printf("Before call : a = %d, b = %d\n", a, b);
    swap(&a, &b);               /* send the addresses */
    printf("After  call : a = %d, b = %d\n", a, b);
    return 0;
}

void swap(int *x, int *y)
{
    int temp = *x;              /* *x is the value at that address */
    *x = *y;
    *y = temp;
}
```

Output

```
Before call : a = 10, b = 20
After  call : a = 20, b = 10
```

Difference between call by value and call by reference

Point	Call by value	Call by reference
What is passed	a copy of the value	the address of the variable

Parameters are	ordinary variables	pointer variables
Effect on originals	they are not changed	they are changed
Memory used	more, because copies are made	less, only addresses are sent
Values returned	only one, through return	more than one can be changed
Default in C	yes	no, it must be written using pointers

10. Recursion

Recursion is the process in which a function calls itself, either directly or through another function. A function written in this way is called a recursive function.

The two parts of a recursive function

- Base case — the condition under which the function stops calling itself and returns a value directly. Without it the calls never stop.
- Recursive case — the function calls itself with a smaller input, which moves the problem closer to the base case.

```
return_type function_name(parameters)
{
    if (base condition)
        return a value;             /* stops the recursion */
    else
        return function_name(smaller input);   /* calls itself */
}
```

Algorithm — factorial using recursion

Step 1: Start
Step 2: Read n
Step 3: If n is 0 or 1, return 1 and stop
Step 4: Otherwise return n multiplied by factorial(n − 1)
Step 5: Print the value returned
Step 6: Stop

Example

```
/* Factorial using recursion */
#include <stdio.h>

int factorial(int n);

int main()
{
    int n = 5;
    printf("Factorial of %d = %d\n", n, factorial(n));
    return 0;
}

int factorial(int n)
```

```
{
    if (n == 0 || n == 1)       /* base case : stops the recursion */
        return 1;
    else
        return n * factorial(n - 1);    /* recursive case */
}
```

Output

```
Factorial of 5 = 120
```

How the calls unfold

Each call waits for the call it made. The calls keep opening until the base case is reached, and then the values are returned back one level at a time.

Figure 11: factorial(4) opens four calls, reaches the base case, and returns the values back

Example — printing every call and return

```
/* Watching the calls open and close */
#include <stdio.h>

int factorial(int n);

int main()
{
    printf("Answer = %d\n", factorial(4));
    return 0;
}

int factorial(int n)
{
    printf("  call  factorial(%d)\n", n);
    if (n == 0 || n == 1)
    {
```

```
        printf("  return 1 (base case)\n");
        return 1;
    }
    int result = n * factorial(n - 1);
    printf("  return %d from factorial(%d)\n", result, n);
    return result;
}
```

Output

```
  call  factorial(4)
  call  factorial(3)
  call  factorial(2)
  call  factorial(1)
  return 1 (base case)
  return 2 from factorial(2)
  return 6 from factorial(3)
  return 24 from factorial(4)
Answer = 24
```

More recursive examples

Example — sum of first n natural numbers

```
/* Sum of first n natural numbers using recursion */
#include <stdio.h>

int sum(int n);

int main()
{
    int n = 5;
    printf("Sum of first %d numbers = %d\n", n, sum(n));
    return 0;
}

int sum(int n)
{
    if (n == 0)
        return 0;
    else
        return n + sum(n - 1);
}
```

Output

```
Sum of first 5 numbers = 15
```

Example — Fibonacci series

```
/* Fibonacci series using recursion */
#include <stdio.h>

int fib(int n);

int main()
{
    int terms = 8, i;

    printf("Fibonacci series : ");
    for (i = 0; i < terms; i++)
```

```
        printf("%d ", fib(i));
    printf("\n");
    return 0;
}

int fib(int n)
{
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fib(n - 1) + fib(n - 2);
}
```

Output

```
Fibonacci series : 0 1 1 2 3 5 8 13
```

Recursion compared with iteration

Point	Recursion	Iteration (loop)
Definition	a function calls itself	a set of statements is repeated
Termination	through the base case	when the loop condition becomes false
Memory	more, every call is stored in the stack	less, no extra calls are made
Speed	slower, because of the repeated calls	faster
Code size	shorter and closer to the definition	usually longer
Risk	stack overflow if the base case is wrong	infinite loop if the update is wrong

Recursion suits problems whose definition is itself recursive, such as factorial, Fibonacci numbers, and the Towers of Hanoi. For simple counting work, a loop is the better choice.

11. Practice Programs

Each program below uses the control structures and functions covered in this unit. In the sample runs, the values shown after each prompt are typed by the user.

Program 1: Sum of even and odd numbers from 1 to n

```
/* Sum of even and odd numbers from 1 to n */
#include <stdio.h>

int main()
{
    int n, i, evenSum = 0, oddSum = 0;

    printf("Enter n : ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
```

```
    {
        if (i % 2 == 0)
            evenSum = evenSum + i;
        else
            oddSum = oddSum + i;
    }

    printf("Sum of even numbers = %d\n", evenSum);
    printf("Sum of odd numbers  = %d\n", oddSum);
    return 0;
}
```

Output

```
Enter n : 10
Sum of even numbers = 30
Sum of odd numbers  = 25
```

A single for loop visits every number, and the if-else inside it decides which total the number should be added to.

Program 2: Check whether a number is prime

```
/* Check whether a number is prime */
#include <stdio.h>

int main()
{
    int num, i, isPrime = 1;

    printf("Enter a number : ");
    scanf("%d", &num);

    if (num <= 1)
        isPrime = 0;

    for (i = 2; i <= num / 2; i++)
    {
        if (num % i == 0)
        {
            isPrime = 0;
            break;              /* no need to check further */
        }
    }

    if (isPrime)
        printf("%d is a prime number\n", num);
    else
        printf("%d is not a prime number\n", num);

    return 0;
}
```

Output

```
Enter a number : 29
29 is a prime number
```

The loop tries every divisor up to num / 2. The moment a divisor is found, break stops the search, because one divisor is enough to prove the number is not prime.

Program 3: Check whether a number is a palindrome

```
/* Check whether a number is a palindrome */
#include <stdio.h>

int main()
{
    int num, temp, digit, rev = 0;

    printf("Enter a number : ");
    scanf("%d", &num);

    temp = num;

    while (temp != 0)
    {
        digit = temp % 10;
        rev   = rev * 10 + digit;
        temp  = temp / 10;
    }

    if (rev == num)
        printf("%d is a palindrome\n", num);
    else
        printf("%d is not a palindrome\n", num);

    return 0;
}
```

Output

```
Enter a number : 12321
12321 is a palindrome
```

The original number is copied into temp before the loop, because the loop destroys the value while reversing it. The reversed value is then compared with the original.

Program 4: Menu driven calculator

```
/* Menu driven calculator using switch inside do-while */
#include <stdio.h>

int main()
{
    int choice, a, b;

    do
    {
        printf("\n1. Add  2. Subtract  3. Multiply  4. Exit\n");
        printf("Enter your choice : ");
        scanf("%d", &choice);

        if (choice == 4)
            break;

        printf("Enter two numbers : ");
        scanf("%d %d", &a, &b);

        switch (choice)
        {
```

```
            case 1 : printf("Result = %d\n", a + b); break;
            case 2 : printf("Result = %d\n", a - b); break;
            case 3 : printf("Result = %d\n", a * b); break;
            default: printf("Invalid choice\n");
        }
    } while (choice != 4);

    printf("Calculator closed\n");
    return 0;
}
```

Output

```
1. Add  2. Subtract  3. Multiply  4. Exit
Enter your choice : 1
Enter two numbers : 8 3
Result = 11

1. Add  2. Subtract  3. Multiply  4. Exit
Enter your choice : 3
Enter two numbers : 4 5
Result = 20

1. Add  2. Subtract  3. Multiply  4. Exit
Enter your choice : 4
Calculator closed
```

The do-while loop shows the menu at least once and keeps repeating it, while the switch carries out the chosen operation. This combination is the standard way of writing a menu driven program.